

Code Assessment of the MetaDEX Coordination Layer Smart Contracts

August 27, 2026

Produced for



Contents

1	Executive Summary	3
2	Assessment Overview	4
3	System Overview	6
4	Trust Model	15
5	System Considerations	18
6	Terminology	19
7	Findings	20
8	Open Questions	34
9	Limitations and use of report	35



1 Executive Summary

Dear Velodrome Team,

Thank you for trusting us to help Velodrome with this security audit. Our executive summary provides an overview of subjects covered in our audit of the latest reviewed contracts of MetaDEX Coordination Layer according to [Scope](#) to support you in forming an opinion on their security risks.

Velodrome implements the governance and token emissions functionality of a multi-chain DEX.

The most critical subjects covered in our audit are [Operational Assumptions](#), voting power accounting, emissions accounting, and cross-chain state drift.

Security regarding all the aforementioned subjects is satisfactory. Regarding cross-chain state drift, note [Leaf Operator Is Not Invalidated upon NFT Transfer](#) and [Root and Leaf Chains Drift After Setting Them Both from Suspended to Active](#).

The general subjects covered are access control, code quality, defense-in-depth, and events.

Security regarding all the aforementioned subjects is improvable.

In summary, we find that the codebase provides an improvable level of security.

Because new code iterations were continuously delivered as the review was ongoing, and due to the design complexity and lack of end-to-end testing, we believe that an above-average number of undiscovered issues remain, as the time allocated wasn't sufficient to cover all subjects comprehensively.

It is important to note that security audits are time-boxed and cannot uncover all vulnerabilities. They complement but don't replace other vital measures to secure a project.

The following sections will give an overview of the system, our methodology, the issues uncovered, and how they have been addressed. We are happy to receive questions and feedback to improve our service.

Sincerely yours,
ChainSecurity

2 Assessment Overview

In this section, we briefly describe the overall structure and scope of the engagement, including the code commit which is referenced throughout this report.

2.1 Scope

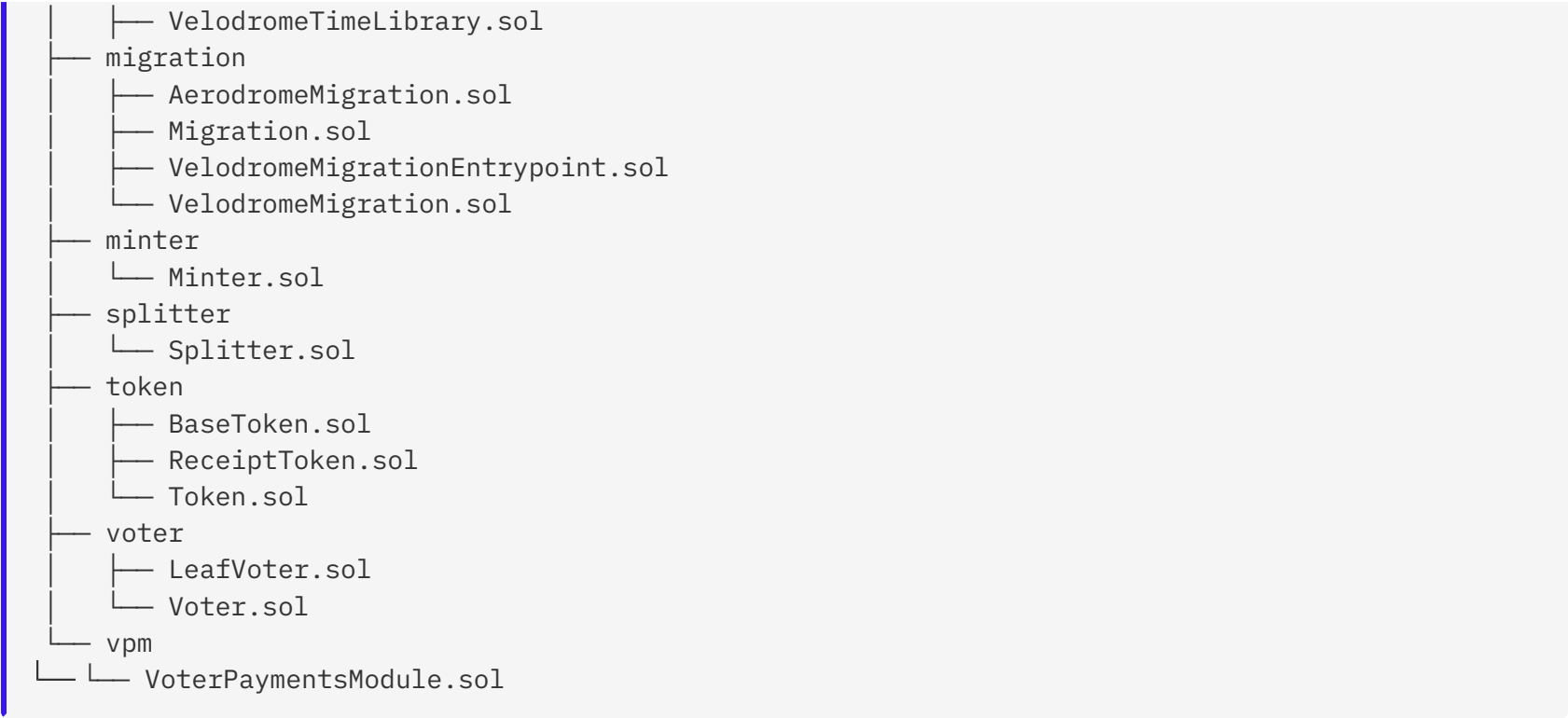
The assessment was performed on the source code files inside the MetaDEX Coordination Layer repository based on the documentation files. The table below indicates the code versions relevant to this report and when they were received.

All 1.x versions were covered by a single continuous review with no fixes round in between. A finding tagged e.g. Version 1.2 may therefore have been present at Version 1.0 as well. The tags only reflect roughly when the issue was found. Further, we are aware of some patches for findings reported as open, but those code versions were not put in-scope for the current report due to time constraints.

Version	Date	Commit Hash	Note
1.0	2 July 2026	67dd9844256030078a6312ffcb6c7bfb07862fb6	First recorded iteration
1.1	10 July 2026	48086ff40e1ba8a8da87dcf20d306883a807d2b3	
1.2	24 July 2026	d00eb70d89c18a87c3849676736feea7edd51377	
1.3	7 August 2026	5109b6409f8eafb9c688530cfb8cbcd4ffe04f64	
1.4	17 August 2026	972882c878dfd5b3096d7a5cdbdaa787f262fb1e	
1.5	19 August 2026	6fd91fe51352fc839a5d02c4065796daddc27a8a	
1.6	25 August 2026	a6b463d76f15f54ad16668030fe870172c8f1699	Intermediate report

For the Solidity smart contracts, the compiler version 0.8.30 was chosen.

```
V3/src
├── access
│   └── GuardedAccessControlEnumerable.sol
├── bridge
│   ├── hyperlane
│   │   └── HyperlaneAdapter.sol
│   ├── LeafMessageOrchestrator.sol
│   ├── MessageOrchestrator.sol
│   ├── RootLocalAdapter.sol
│   └── RootMessageOrchestrator.sol
├── core
│   └── VotingEscrow.sol
├── handlers
│   ├── LeafEmissionsHandler.sol
│   └── RootEmissionsHandler.sol
├── libraries
│   ├── BalanceLogicLibrary.sol
│   ├── CheckpointLogicLibrary.sol
│   ├── DelegationLogicLibrary.sol
│   ├── ProtocolConstants.sol
│   ├── Roles.sol
│   └── SafeCastLibrary.sol
```



2.1.1 Excluded from scope

All other files are out-of-scope. OpenZeppelin and Solady are assumed correct. Files reviewed as part of the *Application Layer* parallel engagement are assumed correct.

3 System Overview

This system overview describes the version `Version 1.6` of the contracts as defined in the [Assessment Overview](#).

Furthermore, in the findings section, we have added a version icon to each of the findings to increase the readability of the report.

Velodrome offers MetaDEX Coordination Layer, the governance and emissions backbone of a multi-chain DEX. Users stake `TOKEN` into a `VotingEscrow` position (`sAERO` , an ERC-721), split that position's voting power across chains, and then across gauges on each chain. Emissions follow the votes: the root chain never sends tokens anywhere, it only meters how much each chain is entitled to, and each chain hands out a locally minted `ReceiptToken` . Holders burn that `ReceiptToken` on the leaf to get `TOKEN` on the root.

The system is split into one single *root* deployment and one *leaf* deployment per supported chain. Root owns the single source of truth: the escrow, the emission schedule, every token's chain-level allocation, and a per-chain emission ceiling. Leaves own the gauge-level distribution: they take the budget and the emission rate root ships them, spread it across their gauges, and cap each gauge locally. The two sides are joined by an orchestrator/adaptor pair that carries typed, nonce-stamped messages over Hyperlane. Because the root chain also hosts a leaf deployment, a dedicated local adaptor delivers root-to-root messages synchronously inside the same transaction.

Root accrues a `ceiling` per chain from the global emission schedule and the chain's share of voting power. Each leaf accrues the same quantity using the same arithmetic. Redeems are checked against root's ceiling.

The gauge implementations, the reward contracts (`VotingRewardsManager` , `FeeDistribution` , `IncentiveStreaming`), the pools, the `GaugeManager` and the `FactoryRegistry` are part of the parallel *Application Layer* engagement and are out of scope here.

3.1 Tokens

`BaseToken` is a shared Solady `ERC20` base: it stores name and symbol, precomputes the EIP-712 name hash used by `permit`, rejects zero-address recipients on `transfer`, `transferFrom` and every `_mint` path (including constructor pre-mints), and disables Solady's implicit infinite Permit2 allowance.

`Token` is the canonical root-chain emissions token. Its constructor pre-mints the two migration allocations to `MIGRATION` (Aerodrome) and `VELODROME_MIGRATION` (Velodrome) and derives `TRANSFERS_ENABLED_AT` as `migrationOpen + 1 week`, with `migrationOpen` required to be a future, week-aligned timestamp. Until that instant, `transfer` and `transferFrom` revert for everyone except the two migration contracts, so migrated balances exist but cannot circulate before the market opens. Minting afterwards is exclusive to the `Minter`; burning is exclusive to the `VotingEscrow` (fee burns) and to the two migration contracts (burning their unused allocation).

`ReceiptToken` is the per-leaf claim on root emissions. It is minted and burned only by the local `LeafVoter`: minted into the chain's `EmissionsHandler` when a gauge claims its accrued share, and burned from the redeemer on the leaf when they redeem. The burn and the payout are on opposite chains: burning the receipt only dispatches the claim, and the matching `TOKEN` is settled on root once the message is delivered.

Unlike `Token`, `ReceiptToken` adds no transfer gate of its own: it inherits `BaseToken`'s `transfer` and `transferFrom` unchanged, so beyond the zero-address recipient check the receipt is a freely transferable ERC-20 (with Solady's `permit`, and without the implicit infinite Permit2 allowance). Receipts therefore circulate on their leaf like any other token, and the party who redeems need not be the LP who earned it. Redemption pays whichever `recipient` the burner names, so the right to the underlying `TOKEN` travels with the receipt balance. The privileged `burn(from, amount)` takes an arbitrary holder and consumes no allowance, but the `LeafVoter` only ever calls it with the redeemer's own address, so a holder's balance can only be burned by that holder's own `redeem` call.

The contracts provide the following functions:

- `Token.mint(to, amount)` : `MINTER` only.
- `Token.burn(amount)` : burns the caller's own balance; callable only by `VOTING_ESCROW`, `MIGRATION` or `VELODROME_MIGRATION`.
- `Token.transfer()` / `transferFrom()` : revert before `TRANSFERS_ENABLED_AT` unless the sender is a migration contract.
- `ReceiptToken.mint(to, amount)` / `burn(from, amount)` : `LEAF_VOTER` only; the burn needs no allowance from `from`, and is only ever invoked with the caller of `LeafVoter.redeem`.
- `ReceiptToken.transfer()` / `transferFrom()` : permissionless and ungated, inherited from `BaseToken`; only the zero-address recipient check applies.¹

3.2 Minter

The `Minter` lives on the root chain and holds the emission rate. It is the only contract allowed to mint `TOKEN`. It does not run a weekly emission loop: it exposes a per-second `emissionRate()` that the `Voter` samples whenever total voting weight moves, and it mints on demand when the `Voter` settles a redeem or spends surplus. Emissions are inactive until `ACTIVATION_TIMESTAMP` (`migrationOpen` + 1 week), before which `emissionRate()` returns zero.

The rate is governed in two layers. A `baseRate` is moved by the `operator` through `setBaseRate`, bounded by `maxBaseRateChangePips` per update and a `baseRateUpdateCooldown` between updates. The pair is additionally cross-validated by `_validateRateControls`, which converts the cooldown into an updates-per-week count and rejects combinations whose compounded weekly effect would exceed `WEEKLY_LOG_BUDGET_PIPS`.

On top of that, a `FED`-appointed `dynamicRateModule` may propose a rate. It is consulted only when a band is configured, that is, when at least one of `bandFloorPips` / `bandCeilingPips` is non-zero; with both unset the clamp collapses onto the base rate and the module is never called at all.

The two band sides are applied in separate branches, selected by comparing the module's answer against the base rate. A proposal strictly below the base rate is raised to the floor, `baseRate - baseRate * bandFloorPips`, so `bandFloorPips` bounds how far the module may cut emissions. A proposal at or above the base rate is lowered to the ceiling, `baseRate + baseRate * bandCeilingPips`, so `bandCeilingPips` bounds how far it may raise them. Both sides are configured by governance through `setRateBand` and are themselves bounded by the immutable caps `MAX_BAND_FLOOR_PIPS` and `MAX_BAND_CEILING_PIPS`. If `bandFloorPips` is set to zero, then any decrease of the `baseRate` is ignored. Similarly, `bandCeilingPips = 0` blocks any increase of the `baseRate`.

Every mint routes a `teamRate` share (at most 5%) to the `Splitter` on top of the requested amount, and `MIN_MINT_AMOUNT` rejects dust mints.

The contract provides the following functions:

- `mint(amount, recipient)` : `VOTER` only; mints `amount` to `recipient` plus the team share to `SPLITTER`.
- `emissionRate()` : view; returns the base rate, optionally overridden by the dynamic module within the configured band.
- `setBaseRate(newRate)` : `operator` only; bounded by the max change and the cooldown.
- `setDynamicRateModule(module)` : `fed` only; the zero address disables the module.
- `setOperator()`, `setFED()`, `setMaxBaseRateChangePips()`, `setBaseRateUpdateCooldown()`, `setTeamRate()`, `setRateBand()`, `setRateModuleGasLimit()` : `GOVERNANCE_ROLE` on the `Voter`.

3.3 Splitter

The `Splitter` receives the team share passively: the `Minter` mints into it with no callback. The contract distributes its tokens across a configured recipient list in proportion to their PIPS shares. Accounting is index-based: `_accrueGlobal` compares the live `TOKEN` balance against `accountedBalance` and folds the difference into `globalAccrualIndex`, so any transfer into the contract, from the `Minter` or from anyone else, is picked up automatically.

Two rounding details are deliberate. `PRECISION` is an exact multiple of `MAX_PIPS`, so the global index absorbs an inflow with no remainder. Per-recipient settlement floors to whole wei, and when the accrual floors to zero the recipient's `lastSettledIndex` is *not* advanced.

Replacing the list settles every outgoing recipient at its old share before clearing it, so pending entitlements survive a reconfiguration and are still claimable afterwards. The new list must be non-empty, at most `MAX_RECIPIENTS` (50) entries, free of duplicates, the zero address and the `Splitter` itself, and its shares must sum to exactly `MAX_PIPS`.

The contract provides the following functions:

- `claim(recipient)`: permissionless; settles and pays out `recipient`'s balance.
- `setRecipients(recipients, shares)`: `SPLITTER_CONFIG_ROLE` on the `Voter`; settles the outgoing list, then installs the new one.
- `earned(recipient)`, `allRecipients()`: views.

3.4 VotingEscrow

The `VotingEscrow` (`sAERO`) escrows `TOKEN` inside ERC-721 positions. A stake is either *decaying*, with a week-aligned `end` at most `MAXTIME` (4 years) out and voting power decaying linearly to zero, or *permanent*, with no end and constant voting power. Global and per-token checkpoints follow the Curve/Solidly model: a `bias / slope` pair per point, scheduled slope reductions keyed by unlock week, plus a separate `permanentStakeBalance` for the non-decaying part. The checkpoint, balance and delegation math live in `CheckpointLogicLibrary`, `BalanceLogicLibrary` and `DelegationLogicLibrary`, linked as external libraries.

The escrow is tightly coupled to the root `Voter`, which mirrors each position onto a per-chain ledger. Deposits and reshapes call `VOTER.parkOnChain0` so freshly created voting power is immediately booked on the idle `CHAIN0` sink and is allocatable; `increaseStakingPeriod` uses the same call with a zero amount purely to re-anchor the position to its new shape. Conversely, `withdraw` and `downgradeFromPermanentStake` require that the position's *entire* amount currently sits on `CHAIN0`: remote voting power must first be returned through the `DEALLOC_GAUGE` sentinel or an emergency deallocation. Root and every leaf agree that the token holds nothing on their chain before its shape changes. `withdraw` additionally calls `VOTER.clearToken` so a later `reviveStake` of the same id starts from a clean ledger.

`tokenId 0` is a protocol-owned accumulator rather than a real NFT. It is never minted, is forced permanent inside `_commit`, collects the `VoterPaymentsModule`'s fees, and is the only position `burnFees` can destroy value from. The burn drops `supply`, burns the underlying `TOKEN` and mirrors the reduction onto the `Voter`.

Another entry point of escrow is the function `rebalanceUnderlying` which is reserved for `VPM_ROLE` holders. It moves stake between positions in one batch: each source is drained (requiring the caller to hold owner authorization for it), and each destination is either the accumulator, a freshly minted position, or an existing position. Two invariants are enforced across the whole call: total in must equal total out, and no destination may unlock earlier than the latest source unlock. The accumulator is exempt from the unlock unlock, and a minted destination inherits the sources' unlock by construction. The batch is mirrored onto the `Voter`'s `CHAIN0` ledger in a single call.

Delegation is permanent-only and delegates the full staked amount; any exit from permanence clears it.

`delegateBySig` rejects upper-half `s` values and enforces a per-signatory nonce and an expiry. A flash-loan guard zeroes `balanceOfNFT` in the same block a token changed owner.

The contract provides the following functions:

- `createStake(value, stakingWeeks, isPermanent)` : permissionless, mints a new position for the caller.
- `reviveStake(tokenId, value, stakingWeeks, isPermanent)` : only owner or approved party; refunds an emptied, expired shell; rejects a still-funded position.
- `increaseStakeAmount(tokenId, value)` / `increaseStakingPeriod(tokenId, stakingWeeks)` : only owner or approved party; increases the staked `TOKEN` amount or the staking duration.
- `withdraw(tokenId, recipient)` : only owner or approved party; expired, non-permanent stakes only, and only when fully parked on `CHAIN0`. Returns the entire `TOKEN` amount locked behind `tokenId` to the `recipient`.
- `upgradeToPermanentStake(tokenId)` / `downgradeFromPermanentStake(tokenId)` : only owner or approved party; the downgrade requires full `CHAIN0` coverage and clears delegation.
- `delegate(delegator, delegatee)` / `delegateBySig(...)` : only owner or approved party; permanent stakes only.
- `rebalanceUnderlying(sources, destinations)` : callable by `VPM_ROLE` and requires owner authorization per source in `sources`.
- `burnFees(amount)` : callable by `BURN_FEES_ROLE`; burns from the `tokenId-0` accumulator.
- `setArtProxy(proxy)` : callable by `ART_PROXY_ADMIN_ROLE`.
- `checkpoint()` : permissionless global checkpoint poke.
- `balanceOfNFT()`, `balanceOfNFTAt()`, `totalVotingPower()`, `getPastVotes()`, `getPastTotalSupply()` : permissionless views.

3.5 VoterPaymentsModule

The `VoterPaymentsModule` (VPM) is meant to be a holder of `VPM_ROLE` on the escrow. It is the user-facing wrapper around `rebalanceUnderlying`: `depositIntoNFT` merges stake from several positions into one destination (an existing id, a fresh mint, or the accumulator), and `withdrawToNFT` splits one source across several destinations. It never holds `TOKEN` itself, and it authorizes every source through `VOTING_ESCROW.isAuthorized`, so the escrow's own ownership rules still apply.

Its purpose is fee collection. Each operation charges a rate in PIPS taken off the top of the moved amount and routed to the `tokenId-0` accumulator as an extra destination leg, where it later becomes `burnFees` material. Rates are per `(selector, caller) : fees[sig][address(0)]` is the default and doubles as a protocol-wide ceiling, so that a caller that was registered with a negotiated rate pays either its negotiated rate or the default rate, whichever is lower. Marking a selector `restricted` rejects callers with no registered entry entirely, turning the operation into a whitelist. `removeFee` cannot delete the default entry.

On withdrawal the destinations are trimmed proportionally to `net / total`, legs that trim to zero are dropped, and the whole residual (the nominal fee plus rounding dust) becomes the fee leg, so the escrow's balance-conservation check always sees an exact match. A withdrawal where every destination trims to zero is rejected, as is minting a fresh position with a zero net amount.

The contract provides the following functions:

- `depositIntoNFT(sources, destinationId, recipient)` : caller must be authorized on every source; `type(uint256).max` as the destination mints a new position.
- `withdrawToNFT(sourceId, destinations)` : caller must be authorized on the source; only a mint leg may name a recipient in `destinations`

- `setFee(sig, caller, rateInPips)`, `removeFee(sig, caller)`, `setRestricted(sig, value)`: callable by `FEE_MANAGER` role.
- `getFee(sig, caller)`: view of the effective rate.

3.6 Voter

The root `Voter` is the hub. It keeps, per veNFT, how much voting power sits on each chain (`allocationChainAmounts`) and the shape (`lastStakeEnd`, `isPermanent`) that power was booked under; per chain, a decaying `Point`, an accrued emission `ceiling`, redeem and surplus accounting and a `ChainStatus`; and globally, `totalPoint`, the accumulators `index / timeIndex` and the emission scalar `emissionsPerVP`.

Allocation. Voting power enters the ledger only through the escrow, which parks it on `CHAIN0`, a virtual sink with no leaf. `allocateChains` then moves budget from `CHAIN0` onto real chains additively: the caller supplies a strictly ascending list of `(chainId, delta)` entries whose sum may not exceed what the token has parked, and each entry ships an `AllocateChain` message carrying the delta, the current `emissionsPerVP` and the live stake snapshot. Movement in the other direction never originates on root: budget returns leaf-first, through the `DEALLOC_GAUGE` sentinel in a gauge vote. `allocateGauges` ships a gauge split for one chain, validated against exactly the amount booked there, and `allocate` composes both in one transaction.

A token has exactly one *shape* across all chains. When the escrow shape changes, `_reanchor` re-books every allocated chain at the new shape before any delta is applied; `allocateGauges`, which cannot re-anchor, instead rejects a mismatch with `StaleShape` and asks the caller to reconcile first.

Metering. `emissionsPerVP` is `emissionRate * PRECISION / totalWeight`, resampled after every change to total weight. `index` integrates that scalar over time and `timeIndex` integrates it weighted by absolute timestamp (stored doubled, so the halving folds into a single division later); both are snapshotted at every week boundary crossed. A chain's ceiling is then accrued one segment per week boundary via `_segmentAccrual`, which prices the end weight against `index` and adds back the weight the segment held before decaying using `timeIndex`.

Redeeming. `processRedeem` is the inflation backstop and the only path that turns leaf receipts into `TOKEN`. Delivered by the orchestrator, it settles the origin chain, applies a max-rule to the leaf's reported undistributed surplus, and requires that `reportedSurplus + totalRedeemed + amount` stays within the chain's accrued `ceiling` (extended, for the payout only, by that chain's `donatedBuffer`). Anyone may donate `TOKEN` into a chain's buffer; buffer-backed payouts transfer existing tokens instead of minting, and reported surplus deliberately does *not* draw on the buffer, since it is a claim on future mints. `CeilingExceeded` is a revert rather than a silent skip: a leaf briefly ahead of root rolls the transport nonce back and is redelivered once accrual catches up. A `Suspended` origin is rejected outright.

Chain lifecycle. Chains are registered explicitly and move between `None`, `Active`, `Paused`, `Suspended` and `Sunset`. `None` is the default state. A chain is set to `Active` upon registration. `Paused` blocks new dispatches while the ceiling keeps accruing. `Suspended` is the kill switch: accrual is diverted to `cumulativeSuspendedSurplus` instead of the ceiling, redeems are rejected, and owners can `emergencyDeallocate` their voting power straight back to `CHAIN0` without the leaf confirming if emergency deallocations are allowed. `Sunset` is an orderly wind-down: it also diverts accrual, accepts no new placement, and leaves only the `DEALLOC_GAUGE` exit open. The transition matrix is constrained so that `Suspended` cannot become `Paused` and `Sunset` can only exit through `Suspended`. Accrual is diverted while suspended or sunset, and each chain's leaf-reported undistributed surplus, is mintable by governance through `spendSurplus`, capped at `spendableSurplus`.

The contract provides the following functions:

- `allocateChains(tokenId, dispatches, refundRecipient)`, `allocateGauges(tokenId, chainId, gauges, gasLimit, refundRecipient)`, `allocate(...)`: owner or approved party on the veNFT; payable, forwarding transport fees.

- `claimRewards(tokenId, params, refundRecipient)`: owner or approved party; dispatches reward claims to leaves.
- `emergencyDeallocate(tokenId, chainId, gasLimit, refundRecipient)`: owner or approved party; for suspended chains with `emergencyDeallocationAllowed` set only.
- `reduceCooldown(tokenId, chainId, reduction, ...)`: callable by `VPM_ROLE` on the escrow; grants a token a one-shot reduction of a leaf's allocation cooldown.
- `donate(chainId, amount)`: permissionless; adds non-minted backing for a chain's redeems.
- `processRedeem(...)`, `processDeallocation(...)`: `ORCHESTRATOR` only.
- `parkOnChain0(tokenId)`, `rebalanceChain0(sources, destinations)`, `burn(amount)`, `clearToken(tokenId)`: `VOTING_ESCROW` only.
- `registerChain()`, `setChainStatus()`, `setEmergencyDeallocationAllowed()`: callable by `CHAIN_CONFIG_ROLE` / `CHAIN_STATUS_ROLE` respectively
- `setMaxAllocationAge()`: callable by `VOTER_CONFIG_ROLE`.
- `spendSurplus()`: callable by `GOVERNANCE_ROLE`.
- `setOperator(tokenId, chainId, operator, ...)`: owner or approved party; names the address allowed to vote locally on that leaf.

3.7 LeafVoter

The `LeafVoter` is deployed on every chain, including root. It converts the chain budget and emission scalar root ships it into per-gauge entitlements, using the same two-accumulator segment arithmetic as root but with a per-gauge cap applied per segment. Every gauge holds a monotonically increasing `ceiling` (its cumulative reward share) that it diffs against its own cursor via the permissionless `settleGauge`; the excess above its per-second `emissionCap`, read from the `FactoryRegistry`, is diverted into `surplusAccrued`, the aggregate the leaf reports back to root on every redeem.

Allocations arrive split in two: `applyChainAllocation` adds the additive budget delta. When the message is the newest chain vote the orchestrator has seen and does not precede an `EmergencyDeallocateMessage`, it refreshes `emissionsPerVP`. `applyGaugeAllocations` distributes the aforementioned budget across gauges. The two are joined by `ZERO_GAUGE (address(0))`, an idle sink with a hardcoded cap of zero that holds any budget not currently sitting on a real gauge. A vote has to name the sink explicitly: the gauge list must sum to exactly the chain budget, and a remainder is rejected rather than assigned to the idle sink.

Budget deltas are additive and reordered budget deltas are applied in their *receiving* order. Gauge votes are newest-wins, gated per token by the orchestrator's nonce bookkeeping. Gauge votes also carry an absolute `expiry` stamped by root, so a message that sat in the transport too long reverts instead of applying a stale split, and a per-token `allocationCooldown` rate-limits re-votes. The cooldown can be shortened by an accumulated, governance-capped reduction granted from root, which is spent only for the shortfall it actually covers.

Beyond the bridged path, an operator named from root may vote locally through `allocateGauges` when governance has enabled `localVotingEnabled`. Local votes use the latest known shape and are subject to the same budget, cooldown and ordering rules.

Emissions leave the leaf when a registered gauge calls `mintEmissions`: the leaf checks the request against `ceiling - claimed - surplus`, mints `ReceiptToken` to the chain's `EmissionsHandler` and lets the handler deliver it. Holders call `redeem` to burn receipts and dispatch a `Redeem` message to root, which carries the leaf's current `surplusAccrued` alongside the amount. A gauge that cannot distribute what it accrued returns it through `forfeitEmissions`, clamped to its own claimable headroom.

The leaf mirrors root's chain-status machine. While `Suspended` or `Sunset` the stored scalar is forced to zero through `_setEmissionsPerVP`, including for messages that land after the flip, so the leaf stops crediting

gauges for exactly the period root diverts to surplus. Incoming root messages keep being applied while suspended. `applyEmergencyDeallocation` clears a token's gauges, cuts its budget by the amount root already drained, and re-parks any survivor on the sink. The contract provides the following functions:

- `applyChainAllocation()`, `applyGaugeAllocations()`, `setOperator()`, `applyCooldownReduction()`, `applyEmergencyDeallocation()` : ORCHESTRATOR only.
- `allocateGauges(tokenId, gauges)` : the token's registered operator (through the root voter), when local voting is enabled.
- `mintEmissions(recipients, amounts)` / `forfeitEmissions(amount)` : registered gauges only.
- `redeem(amount, recipient, gasLimit, refundRecipient)` : permissionless; burns the caller's `ReceiptToken` and dispatches the corresponding `Redeem` message to the root.
- `claimRewards(tokenId, recipient, feeClaims, incentiveClaims)` : the orchestrator or the token's operator; individual claims that fail are skipped and reported by event rather than reverting the batch.
- `settleGauge(gauge)` / `projectedCumulativeRewardShare(gauge)` : permissionless settle and its read-only projection.
- `registerGauge(gauge, activate)` / `activateGauge(gauge)` : GAUGE_MANAGER only.
- `setLocalVotingEnabled()`, `setAllocationCooldown()`, `setMaxAccumulatedCooldownReduction()`, `setMaxGauges()` : callable by `VOTER_CONFIG_ROLE`.
- `setCanVoteForZeroCapGauges()` : callable by `TOKEN_WHITELIST_ROLE`.
- `setChainStatus()` : callable by `CHAIN_STATUS_ROLE`.

3.8 Cross-Chain Messaging

`MessageOrchestrator` is the shared base and owns the wire format: a 39-byte header of `(uint8 msgType, uint256 chainNonce, uint48 dispatchedAt)` prepended to the body. Decoding rejects short payloads, the `None` type, out-of-range types, and any message whose `dispatchedAt` lies in the receiver's future (`ClockBehindDispatch`).

`RootMessageOrchestrator` keeps one adapter, one outbound nonce stream and one used-nonce set per leaf chain. `dispatch` is callable only by the `Voter`; it resolves every adapter, retained cost and transport fee before the first send, since a send hands control to the refund recipient. When a dispatch carries a deallocation return, the orchestrator withholds that chain's configured `deallocationReturnCost` from the forwarded value and keeps it locally, so the root side pre-pays the leaf's return message. Inbound, `route` accepts only the registered adapter for the origin chain, consumes the nonce single-use, and forwards `Redeem` and `Deallocate` to the `Voter`; every other type is rejected. A `Deallocate` is accepted even from a suspended origin, and the credit clamps to what the token actually books.

`LeafMessageOrchestrator` is the mirror image with a single adapter pointed at root, whose `REMOTE_CHAIN_ID` must equal the configured root chain id. It also holds the pre-funding pool that pays for deallocation returns. A return is emitted whenever a gauge vote carries the `DEALLOC_GAUGE` sentinel, and who pays for it depends on how that vote arrived. A vote bridged from root carries no native value into the leaf, so the orchestrator covers the fee out of its own balance and names itself as the transport's refund address, which returns any unspent remainder to the pool. A vote made locally by the token's operator is payable, so the operator's attached value is passed on to the adapter in full and the operator is named as the refund address, leaving it to the transport to hand back whatever the fee did not consume. In both cases the fee is a live `quoteMessage` against the adapter rather than a stored constant, and an underfunded return reverts with `InsufficientDeallocationMessageValue` rather than failing deeper in the transport. Inbound, it applies the ordering policy per message type: additive for chain allocations and cooldown reductions, newest-wins per token for gauge votes, operator updates and shape refreshes, and highest wins for emergency deallocations, whose nonce also suppresses pre-drain chain deltas. Handlers are called without `try / catch` on purpose: a failing handler rolls back the nonce. The failing message can be retried.

`HyperlaneAdapter` is responsible for cross-chain message delivery. It is bound at construction to exactly one remote chain id and Hyperlane domain, rejects a remote domain equal to the local one, and only `ADAPTER_CONFIG_ROLE` on the local voter may set the counterpart address. `sendMessage` accepts only the orchestrator and requires a non-zero gas limit; `handle` accepts only the mailbox and only messages from the configured origin domain and remote adapter. `quoteMessage` prices the exact metadata `sendMessage` will use, so the quote cannot understate the fee.

`RootLocalAdapter` exists only on root, where both a root and a leaf deployment coexist. It has no transport: `sendMessage` accepts zero value, dispatches synchronously from whichever of the two orchestrators called it into the other, and `handle` and `setRemoteAdapter` revert.

3.9 Emissions Handlers

Both handlers are bound to a single `LeafVoter` at construction and accept `handleEmissions` from nobody else.

`LeafEmissionsHandler` is the ordinary case: it receives the aggregate `ReceiptToken` the leaf just minted and forwards each leg to its recipient, who holds the receipt locally until they burn it through the leaf's `redeem` and receive `TOKEN` on root.

`RootEmissionsHandler` covers the root-hosted leaf. When `handleEmissions` is called, the handler immediately calls `LeafVoter.redeem` per recipient, which burns the receipt and settles through the local adapter into `Voter.processRedeem` in the same transaction. Note that `redeem` reverts below `MIN_REDEEM_AMOUNT`. Sub-floor legs are deferred rather than reverted: the backing receipt stays in the handler and the amount accrues in `pendingRedeems` until the recipient's total clears the floor, at which point the whole total is redeemed at once.

3.10 Migration

`Migration` is the shared engine for converting v2 positions into v3 ones. Its constructor creates a single permanent v2 veNFT that every migrated position is merged into, so the protocol accumulates the migrated v2 voting power in one place. `depositVeNFT` accepts only a `NORMAL`, unvoted, non-restricted v2 veNFT owned by the caller, unlocks it if permanent, merges it into the migration veNFT, and settles the equivalent v3 position; `depositLiquid` does the same for plain v2 tokens by locking them into the migration veNFT. Both are open only from `MIGRATION_OPEN` onward and are pausable. `_resolveStake` maps the v2 lock onto v3: a permanent lock stays permanent, an expired one settles as liquid `TOKEN`, and an active one settles as a v3 stake whose duration is the remaining time rounded up to whole weeks and clamped to `MAXTIME`.

The owner also wields the migration veNFT's v2 powers: `vote` and `resetVote` cast and clear its v2 gauge vote, while `claimFees` and `claimIncentives` collect its v2 rewards to an owner-chosen recipient.

Separately, `Migration` implements the v2 `EpochGovernor` interface, which is how v2 adjusts its tail emission rate. The v2 `Minter.nudge()` reads the epoch governor's `result()` and moves the tail rate one basis point up on `Succeeded`, one down on `Defeated`, or leaves it unchanged otherwise, at most once per epoch and only within the tail schedule. `Migration` seeds its own `result` to `Defeated` at construction, so once v2 governance installs it as the epoch governor (an external precondition, outside this scope) the owner's `decreaseTailEmissionRate` becomes a per-epoch ratchet winding v2 emissions down: it calls `nudge()` and reverts with `TailEmissionRateNotDecreased` unless the rate genuinely fell. `setResult` lets the owner report a different outcome, so the direction of that ratchet ultimately stays under owner control.

`AerodromeMigration` is the same-chain case on Base. It converts at 1:1, is funded by a pre-mint held in the contract and checks each conversion against that remaining balance, settles stakes by creating the v3 position and transferring it to the recipient, and rejects native value. Its owner may kill and revive v2 gauges, burn the unused allocation once paused, and can only renounce ownership when paused with a zero balance. Claimed

reward tokens are forwarded to a recipient, with `V3_TOKEN` explicitly rejected as a reward token so the migration budget cannot be drained through the reward path.

`VelodromeMigration` is the cross-chain case: it runs on Optimism, converts VELO at a fixed 5.5% ratio, and holds no v3 tokens at all. Instead of settling locally it dispatches a packed settlement message to root through the Hyperlane mailbox, with the caller's value paying the fee and `quoteDepositVeNFT` / `quoteDepositLiquid` exposing the required amount. Its owner also holds the v2 emergency-council powers for root and leaf gauges and can repoint the v2 root voting-rewards recipient. Reward forwarding is restricted to reward contracts that report the Optimism chain id.

`VelodromeMigrationEntrypoint` is the root-side settler. It accepts messages only from the mailbox, only from the Optimism domain, and only from its own address on the remote side, consumes each nonce once, checks the amount against its remaining budget, and then either transfers liquid `TOKEN` or creates a v3 stake and hands it to the recipient. Since delivery is asynchronous, it re-clamps the requested duration to the maximum acceptable at delivery time. Its owner may burn the unused allocation, and renouncing ownership requires a zero balance.

3.11 Access Control

Most contracts use OpenZeppelin `AccessControlEnumerable` through `GuardedAccessControlEnumerable`, which adds one rule: the last holder of a *self-administering* role cannot be removed, by revocation or renunciation. Roles administered by another role stay fully revocable.

`Roles` centralizes the identifiers shared by the `Voter` and `LeafVoter`.

On root, `GOVERNANCE_ROLE` administers itself, `SPLITTER_CONFIG_ROLE`, and `CONFIG_ADMIN_ROLE`, which in turn administers the operational roles (`VOTER_CONFIG_ROLE`, `CHAIN_CONFIG_ROLE`, `CHAIN_STATUS_ROLE`, `ADAPTER_CONFIG_ROLE`, `NATIVE_WITHDRAWER_ROLE`, `RELAY_DEPLOYER_ROLE`, `FACTORY_REGISTRY_ADMIN_ROLE`).

On each leaf the same shape repeats: `GOVERNANCE_ROLE` administers itself, `EMERGENCY_COUNCIL_ROLE`, and `CONFIG_ADMIN_ROLE`, which in turn administers `VOTER_CONFIG_ROLE`, `CHAIN_STATUS_ROLE`, `ADAPTER_CONFIG_ROLE`, `NATIVE_WITHDRAWER_ROLE`, `FACTORY_REGISTRY_ADMIN_ROLE`, `TOKEN_WHITELIST_ROLE`, `MODULE_ADMIN_ROLE` and `GAS_CONFIGURER_ROLE`. A leaf carries no `CHAIN_CONFIG_ROLE`, `RELAY_DEPLOYER_ROLE` or `SPLITTER_CONFIG_ROLE`, since chain registration, relay deployment and the team share are root-only concerns. Only the governor, config admin, adapter authority and emergency council are granted at construction. In both deployments `DEFAULT_ADMIN_ROLE` is deliberately left ungranted, so the OpenZeppelin master slot is empty.

On `VotingEscrow`, `VPM_ADMIN_ROLE` administers itself and `VPM_ROLE`, `BURN_FEES_ADMIN_ROLE` administers itself and `BURN_FEES_ROLE`, and `ART_PROXY_ADMIN_ROLE` administers only itself, holding `setArtProxy` directly rather than gating a separate operable role. `VPM_ROLE` and `BURN_FEES_ROLE` start out unheld and are granted afterwards. The `VoterPaymentsModule` repeats the pattern with one branch, `FEE_MANAGER_ADMIN_ROLE` administering itself and `FEE_MANAGER`, and again only the admin is granted at deployment.

The migration contracts use `Ownable` instead of roles, with `renounceOwnership` overridden to require a paused contract and, a zero balance in case v3 tokens are held.

4 Trust Model

4.1 Fully Trusted Roles

Roles in this category have absolute authority over core protocol parameters, direct custody or movement of protocol/user funds, minting/burning mechanisms, or the ability to appoint/revoke other highly-privileged roles.

Role Name	Associated Contracts	Key Permissions
<code>GOVERNANCE_ROLE</code> / <code>governor</code>	<code>Voter</code> , <code>LeafVoter</code> , <code>Minter</code>	• Can call <code>spendSurplus</code> to mint up to the chain's unspent accrued surplus • Appoints/revokes <code>CONFIG_ADMIN_ROLE</code>, <code>SPLITTER_CONFIG_ROLE</code> and <code>EMERGENCY_COUNCIL_ROLE</code>. • Controls critical inflationary parameters in <code>Minter</code> (e.g., team rates, rate bands, and operator cooldowns). • Sets the <code>fed</code> in <code>Minter</code>, which controls the dynamic rate module.
<code>CONFIG_ADMIN_ROLE</code>	<code>Voter</code> , <code>LeafVoter</code>	• Administers and appoints critical operational roles: <code>VOTER_CONFIG_ROLE</code>, <code>CHAIN_CONFIG_ROLE</code>, <code>CHAIN_STATUS_ROLE</code>, <code>ADAPTER_CONFIG_ROLE</code>, <code>NATIVE_WITHDRAWER_ROLE</code>, <code>RELAY_DEPLOYER_ROLE</code>, <code>TOKEN_WHITELIST_ROLE</code> and <code>MODULE_ADMIN_ROLE</code>.
<code>owner</code> (Migration)	<code>Migration</code> , <code>AerodromeMigration</code> , <code>VelodromeMigration</code> , <code>VelodromeMigrationEntrypoint</code>	• Can halt, pause, and resume migrations. • Can call <code>burnRemaining()</code> to destroy leftover token supply. • Casts and resets governance votes on legacy chains, set migration results, and claims accumulated legacy fees/incentives. • Kills and revives legacy v2 gauges on the root and leaf chains, and redirects legacy leaf reward recipients (<code>setRecipient</code>).
<code>BURN_FEES_ADMIN_ROLE</code>	<code>VotingEscrow</code>	• Administers and appoints the <code>BURN_FEES_ROLE</code>.
<code>FEE_MANAGER_ADMIN_ROLE</code>	<code>VoterPaymentsModule</code>	• Administers and appoints the <code>FEE_MANAGER</code> role.



4.2 Partially Trusted Roles

Roles in this category have restricted power to manage specific operational parameters. While they can alter protocol behavior, their influence is constrained by immutable validation logic, cooldown timers, or absolute limits (caps) built into the code.

Role Name	Associated Contracts	Key Permissions
<code>operator</code>	<code>Minter</code>	Authorized to adjust the inflation base rate (<code>setBaseRate</code>).
<code>fed</code>	<code>Minter</code>	Can configure the dynamic interest rate module address (<code>setDynamicRateModule</code>).
<code>FEE_MANAGER</code>	<code>VoterPaymentsModule</code>	Sets fee rates per operation and restricts operations to whitelisted addresses.
<code>CHAIN_STATUS_ROLE</code>	<code>Voter</code> , <code>LeafVoter</code>	- Can flip chain states (Active, Suspended, Sunset). - Toggles whether emergency deallocations are permitted.
<code>VOTER_CONFIG_ROLE</code>	<code>Voter</code> , <code>LeafVoter</code>	Configures operational boundaries (e.g., <code>setMaxAllocationAge</code> , <code>setLocalVotingEnabled</code> , allocation cooldowns, maximum gauges per user).
<code>VPM_ADMIN_ROLE</code>	<code>VotingEscrow</code>	Administers the <code>VPM_ROLE</code> (can grant or revoke VPM authority to contracts).
<code>VPM_ROLE</code>	<code>VotingEscrow</code>	- Granted to authorized <code>VoterPaymentsModule</code> contracts and can be granted to arbitrary addresses by <code>VPM_ADMIN_ROLE</code> . - Authorized to call <code>rebalanceUnderlying</code> to programmatically adjust staked escrow balances (amounts, lock durations, and NFT destinations), only with the consent of token holders.
<code>CHAIN_CONFIG_ROLE</code>	<code>Voter</code>	Registers new target chains for cross-chain emissions (<code>registerChain</code>).
<code>ADAPTER_CONFIG_ROLE</code>	<code>Voter</code> , <code>LeafVoter</code> , <code>LeafMessageOrchestrator</code> , <code>RootMessageOrchestrator</code> , <code>HyperlaneAdapter</code>	Configures message routing adapters on orchestrator contracts and the remote adapter on <code>HyperlaneAdapter</code> .
<code>TOKEN_WHITELIST_ROLE</code>	<code>LeafVoter</code>	Whitelists token IDs to allow voting on gauges with zero emission caps.
<code>BURN_FEES_ROLE</code>	<code>VotingEscrow</code>	Calls <code>burnFees</code> to burn accumulated fees in <code>tokenId-0</code> .
<code>ART_PROXY_ADMIN_ROLE</code>	<code>VotingEscrow</code>	

Role Name	Associated Contracts	Key Permissions
		• Can change the rendering proxy (<code>setArtProxy</code>) that generates SVGs/ metadata for sAERO NFTs.
<code>NATIVE_WITHDRAWER_ROLE</code>	<code>Voter</code> , <code>LeafVoter</code> , <code>LeafMessageOrchestrator</code> , <code>RootMessageOrchestrator</code>	• Withdraws accumulated native gas balances from orchestrators.
<code>GAS_CONFIGURER_ROLE</code>	<code>LeafVoter</code> , <code>LeafMessageOrchestrator</code>	• Configures leaf-side messaging gas parameters.

4.3 Untrusted Roles

This category consists of standard users, individual token owners, operators, and public participants. They have zero administrative power and are strictly limited to their own assets.

Role Name	Associated Contracts	Key Permissions
<code>Token Operator</code> / <code>User</code>	<code>Voter</code> , <code>LeafVoter</code> , <code>VotingEscrow</code> , <code>VPM</code>	• Can allocate voting weights for their specific token IDs. • Can deposit, withdraw, and delegate their own assets.

4.4 Operational Assumptions

Privileged operations whose safety depends on the sequencing of administrative actions rather than on on-chain enforcement are out of scope of this review. In particular, removing or replacing bridge adapters, and suspending or sunseting a chain while messages are still in flight, are assumed to be carried out correctly. Velodrome is expected to administer such operations according to the procedures documented on `IVoter.setChainStatus`.

4.5 External Dependencies

Hyperlane is assumed to promptly and correctly relay cross-chain messages. The root chain is assumed to function properly. Leaf chains are partially trusted, but the impact of a compromised leaf chain is limited to assets on that chain and emissions up to the ceiling.

5 System Considerations

We leverage this section to highlight findings that are not necessarily issues. The mentioned topics serve to clarify or support the report, but do not require a modification inside the project. Instead, they should raise awareness in order to improve the overall understanding for users and developers.

SC1 Geometric Progression in Rate Limiting

System Consideration	Version 1.2	
----------------------	-------------	--

The rate limiting in `setBaseRate()` has asymmetric behavior due to the formula used: suppose the max base rate change is 99.9%: it takes one cooldown step to reduce the rate from 1000 to 1, but 10 steps `( ceil(log(1000/1, base=1+99.9%)) )` to go back up from 1 to 1000.

SC2 Leaf Operator Is Not Invalidated upon NFT Transfer

System Consideration	Version 1.2	
----------------------	-------------	--

`VotingEscrow._update()` never invalidates the leaf-side operator: Operator authorization is token-scoped yet unrevocable at transfer time, and the leaf has no notion of owner, so a transfer silently hands the buyer an NFT with a live claim delegation they cannot see or atomically clear.

6 Terminology

For the purpose of this assessment, we adopt the following terminology. To classify the severity of our findings, we determine the likelihood and impact (according to the CVSS risk rating methodology).

- **Likelihood** represents the likelihood of a finding to be triggered or exploited in practice
- **Impact** specifies the technical and business-related consequences of a finding
- **Severity** is derived based on the likelihood and the impact

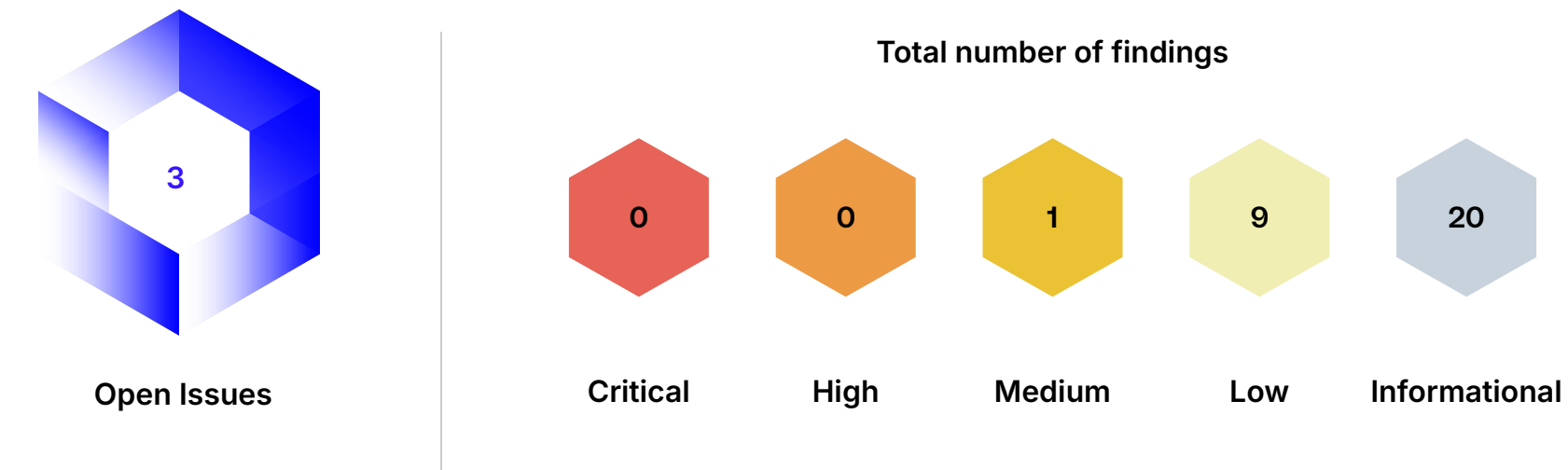
We categorize the findings into four distinct categories, depending on their severity. These severities are derived from the likelihood and the impact using the following table, following a standard risk assessment procedure.

Likelihood	Impact		
	High	Medium	Low
High	Critical	High	Medium
Medium	High	Medium	Low
Low	Medium	Low	Low

As seen in the table above, findings that have both a high likelihood and a high impact are classified as critical. Intuitively, such findings are likely to be triggered and cause significant disruption. Overall, the severity correlates with the associated risk. However, every finding's risk should always be closely checked, regardless of severity.

7 Findings

In this section, we describe the findings identified during the course of the engagement. The findings are categorized by severity as explained in the Terminology section. Below we provide an overview of the total number of findings per severity, as well as the number of open issues.



7.1 Overview

The following table lists all identified findings along with their severity and current status. A finding is considered resolved once it has been fully corrected or the specification has been changed accordingly. Non-informational findings with any other status, including partial fixes, acknowledgements, and accepted risks, remain open.

Medium	#001	cooldownReduction Is Not Cleared at the End of applyTokenAllocation	Code Corrected	✓
Low	#002	tokenGauge() Can Return Stale Value	Code Corrected	✓
Low	#003	Asymmetric Suspension Checks in the Message Orchestrators	Code Corrected	✓
Low	#004	Expired Stake with Remote Allocations Cannot Be Withdrawn Directly	Code Corrected	✓
Low	#005	Leaf Can Pre-Settle Past tAct and Lock in a Stale Rate	Code Corrected	✓
Low	#006	Not All Contracts Inherit from GuardedAccessControlEnumerable	Code Corrected	✓
Low	#007	Paused Chain Locks a Token Out of Voting	Code Corrected	✓
Low	#008	Per-Token Local Voting Remains Enabled	Code Corrected	✓
Low	#009	Potentially Negative Bias Erodes the Permanent Balance	Code Corrected	✓
Low	#010	withdrawToNFT: Entire Withdrawal Can Be Routed to the Fee Accumulator When All Destinations Trim to Zero	Code Corrected	✓
Info	#011	getPastVotes() Accepts Future Timestamps	Code Corrected	✓
Info	#012	MAX_BASE_RATE_CHANGE_PIPS Naming	Code Corrected	✓
Info	#013	Base Contract for Token and ReceiptToken	Code Corrected	✓
Info	#014	ChainStatus Default Value	Code Corrected	✓
Info	#015	Empty Rebalance Emits an Empty Event	Acknowledged	⊖
Info	#016	ERC721Enumerable Compatibility	Code Corrected	✓
Info	#017	Gas Optimisations	Code Corrected	✓
Info	#018	Griefing rebalanceChain0 by Reshaping the Destination Stake	Code Corrected	✓
Info	#019	Incorrect Unit in _creditDeallocation Comment	Code Corrected	✓
Info	#020	Misleading Argument Name	Open	○
Info	#021	Misleading Comment in _allocationChainIds()	Code Corrected	✓

Info	#022	Misleading Recipient in Rebalance Events	Code Corrected	✓
Info	#023	OP Mainnet Naming	Open	○
Info	#024	Redundant Event Emissions	Code Corrected	✓
Info	#025	Restricted Guard in removeFee	Code Corrected	✓
Info	#026	Revoking the FED Freezes the Dynamic Rate Module	Acknowledged	⊖
Info	#027	Role Admins Can Be Revoked	Code Corrected	✓
Info	#028	Root and Leaf Chains Drift After Setting Them Both from Suspended to Active	Acknowledged	⊖
Info	#029	Unused Flash NFT Guard in balanceOfNFT	Open	○
Info	#030	Zero Amounts in rebalanceDelta	Acknowledged	⊖

7.2 Descriptions

#001 `cooldownReduction` Is Not Cleared at the End of `applyTokenAllocation`

Security	Medium	Version 1.2	Code Corrected 
----------	--------	-------------	--

When using transient variables, it is considered best practice to reset them at the end of the call where they are first set. Failure to do so can lead to DoS of subsequent calls, or, conversely, double-counting of the value stored in the variable.

In the specific case of `cooldownReduction`, we cannot determine if this is an issue or intended behavior since the functionality is not clearly documented and not yet implemented.

Code Corrected

The transient variable was removed.

#002 `tokenGauge()` Can Return Stale Value

Correctness	Low	Version 1.2	Code Corrected 
-------------	-----	-------------	--

`Voter.tokenGauges()` can return a stale non-zero value if local voting is enabled. As a matter of accessor discipline, we think accessors shouldn't return garbage data.

Code Corrected

The accessor was removed due to changes to the Voter: it keeps track of neither the gauges nor whether local voting is enabled.

#003 Asymmetric Suspension Checks in the Message Orchestrators

Design	Low	Version 1.2	Code Corrected 
--------	-----	-------------	--

The two message orchestrators apply inconsistent rules for suspended chains. `LeafMessageOrchestrator` gates on its own chain status. Its `route()`, `dispatch()` and `retryCheckpointReward()` all revert with `RouteSuspended` while `LEAF_VOTER.chainStatus()` is `Suspended`. `RootMessageOrchestrator` applies no such rule on the outbound path. Its `dispatch()` does not validate the status of the destination chain, and its `route()` only checks the origin.

Combined with the full exit from a suspended chain described in [Paused Chain Locks a Token Out of Voting](#), this diverges the two ledgers. When governance has set the same status on both chains, the root applies the deallocation and releases the voting power. On arrival the message is rejected, either by `LeafMessageOrchestrator.route()` or behind it by the `chainNotSuspended` modifier on `LeafVoter.applyChainState()` and `LeafVoter.applyTokenAllocation()`. The leaf keeps the stale booking for the token while the root has already moved that voting power elsewhere.

Code Corrected

Suspension policy was moved out of both orchestrators and into the two `Voter` contracts. `RootMessageOrchestrator.dispatch()` still carries no destination status check, but it is reachable only from `Voter`, where every path is gated beforehand. The blanket gate on `LeafMessageOrchestrator` was

removed as well. Root messages now keep applying while the leaf is suspended so that the leaf repairs itself, and `LeafVoter` masks the applied `emissionsPerVP` to zero for that window so no unfunded emissions accrue.

#004 Expired Stake with Remote Allocations Cannot Be Withdrawn Directly

Design	Low	Version 1.5	Code Corrected
--------	-----	-------------	----------------

`VotingEscrow.withdraw` takes an sAERO that is fully parked on `CHAIN0`. `Voter.allocateChains` requires a live sAERO, meaning one that is neither withdrawn nor expired, and reverts with `StakeExpired` otherwise.

If a token's lock expires while its voting power is still allocated to a leaf chain, its owner cannot use either of these functions. They cannot withdraw because the voting power is in use, and they cannot allocate because the stake is expired.

Two paths out of this state exist:

1. The owner relocks the token for an additional week, the minimum possible duration. `increaseStakingPeriod` accepts an expired stake, so the token becomes live again and `Voter.allocateChains` can then deallocate from every leaf chain.
2. The owner calls `Voter.allocateGauges`, which reads the stake as-is rather than requiring it live, and allocates only to `DEALLOC_GAUGE`. This has to be repeated for every chain holding a non-zero allocation.

The second path works only incidentally, as a consequence of `allocateGauges` not requiring a live stake, and neither path is indicated by the reverts the owner encounters.

Code Corrected

The sentinel return is now an explicit exit rather than an incidental one. `Voter.allocateGauges` derives `_deallocOnly` as a list holding exactly one entry that is the `DEALLOC_GAUGE` sentinel, and rejects an expired non-permanent stake unless that gate passes:

```
bool _deallocOnly = _gauges.length == 1 && _hasDeallocSentinel;
if (!_deallocOnly && !_snapshot.isPermanent && _snapshot.stakeEnd <= block.timestamp)
    revert StakeExpired();
```

An expired stake may therefore return its whole budget and do nothing else. The rule is documented on `IVoter.allocateGauges`, and `VotingEscrow` now names the `DEALLOC_GAUGE` return as the way to free remote budget before a withdrawal.

#005 Leaf Can Pre-Settle Past tAct and Lock in a Stale Rate

Correctness	Low	Version 1.2	Code Corrected
-------------	-----	-------------	----------------

The root anchors each chain-state update at `tAct = block.timestamp + latency` through `Voter._computeTAct()` and books the chain's ceiling at the new rate from `tAct` onward. The leaf applies the update through `LeafVoter.applyChainState()`, which settles the accrual with `_settleIndex(_tAct)` before overwriting `chainRate` and `chainWeight`. `_settleIndex` returns early whenever `_tAct <= lastSettlement`.

`lastSettlement` is advanced to `block.timestamp` by the permissionless `settleGauge()`, and likewise by the local `vote()` and by `setChainStatus()`. Any such call in the window between `tAct` and the actual

delivery of the message pushes `lastSettlement` past `tAct`, so the settlement inside `applyChainState()` silently no-ops. The leaf has by then already credited that window at the stale rate and weight, and the arriving message can no longer correct it, while the root has booked the same window at the new rate.

Code Corrected

The activation timestamp was removed. `latency` and `_computeTAct()` no longer exist on the `Voter`, and `LeafVoter._settleIndex()` takes no argument and always settles up to `block.timestamp`.

#006 Not All Contracts Inherit from `GuardedAccessControlEnumerable`

CorrectnessLowVersion 1.2Code Corrected ✓

Only `VotingEscrow` and `VoterPaymentsModule` are inheriting from `GuardedAccessControlEnumerable` instead of from `AccessControlEnumerable`.

Code Corrected

`Voter` and `LeafVoter` now also inherit from `GuardedAccessControlEnumerable`.

#007 Paused Chain Locks a Token Out of Voting

DesignLowVersion 1.2Code Corrected ✓

`Voter._revertIfScopePaused()` rejects a vote when any chain in the dispatch scope is `Paused`. The scope built by `_buildScopeAndDispatches()` must cover every chain the token holds an allocation on, not only the chains the caller is changing. A single paused chain therefore blocks the token from voting at all, including from reallocating its voting power away from that chain, until governance moves the chain out of `Paused`.

The same lockout does not apply to a suspended chain. `_validateAllocations()` calls `_requireChainActive()` on every entry the caller supplies, so a suspended chain cannot be kept or added, but a removal never appears in the caller's allocations and `_revertIfScopePaused()` only tests for `Paused`. A token holder can consequently pull their entire voting power out of a suspended chain and re-vote, while a paused chain leaves them with no move at all.

Code Corrected

The allocation model was redesigned. Allocations are now additive and partial through `allocateChains()`, so a call touches only the chains it lists and no longer has to cover the token's full allocation set. The scope union and `_revertIfScopePaused()` were removed, and every dispatching entrypoint gates its own destination through `_requireChainActive()` or `_requireDispatchableChain()`. Reductions no longer run through the allocation path at all. They are leaf-first, originating on the `LeafVoter` and credited on the root by `Voter.processDeallocation()`, or through `Voter.emergencyDeallocate()` for a suspended chain.

#008 Per-Token Local Voting Remains Enabled

Security	Low	Version 1.2	Code Corrected 
----------	-----	-------------	--

Suppose 1) governance enables local voting for a chain, 2) a token holder enables local voting on a token on their token for that chain, and 3) governance disables local voting. As far as the root `Voter` is concerned, local voting can remain enabled on the token for as long as the owner desires. Note that it is still necessary to enable and possible to disable local voting on the leaf voter directly, and it is more effective.

Code Corrected

`Voter` no longer tracks local voting, only `LeafVoter` has a `localVotingEnabled` toggle, and enforces it on every call to `allocateGauges()`

#009 Potentially Negative Bias Erodes the Permanent Balance

Correctness	Low	Version 1.4	Code Corrected 
-------------	-----	-------------	--

`LeafVoter.applyChainAllocation` books each incoming chain allocation with its own `_applyContribution(ZERO_GAUGE, _allocationDelta, ...)` call and adds the amount to the token's stored allocation. `_processAllocation` later reverses the whole position with a single `_unwindContribution(ZERO_GAUGE, _priorZeroGauge, ...)` call carrying the accumulated total.

That is only sound if `_contribution` is additive in `_allocated`, which it is not. It derives the slope by integer division by `MAXTIME`, so the sum of the per-park slopes is at most the slope of the sum, and strictly less once the individual remainders accumulate past `MAXTIME`. The single unwind therefore removes more slope and more bias than the parks ever added. Because `_unwindContribution` subtracts into signed fields with no lower bound, the point's `bias`, its `slope`, and the `gaugeSlopeChanges` entry for that `stakeEnd` all go negative.

Ordinary gauges are unaffected, since `_processGauge` stores the allocation absolutely and its unwind always carries the same amount its apply did.

The corruption does not clear at expiry. `_walkGauge` floors the negative slope at the first weekly boundary, freezing the bias rather than letting it recover, and the surviving negative slope-change entry later produces a spurious positive slope that the floor accepts.

`_walkGauge` and `_settleGauge` then resolve gauge weight as the raw sum `permanentStakeBalance + bias`, whereas root's `_weightOf` floors the bias at zero before adding it. As a result, while the bias is negative but smaller in magnitude than the permanent balance, the discrepancy is subtracted directly from `permanentStakeBalance`.

Code Corrected

Velodrome independently identified and fixed this issue in parallel during the engagement.

The park is now swapped rather than accumulated. `_swapZeroGaugePark` unwinds the whole prior park and applies the whole new total on every chain allocation, so the sink holds `contribution(total)` at all times and the aggregate unwind in `_processAllocation` cancels exactly.

#010 withdrawToNFT: Entire Withdrawal Can Be Routed to the Fee Accumulator When All Destinations Trim to Zero

Correctness	Low	Version 1.0	Code Corrected
-------------	-----	-------------	----------------

In `withdrawToNFT`, each destination receives `amount * _net / _totalOut` rounded down. It is possible that `_totalOut > 0` yet every individual trim floors to zero. In that case `_nonZeroDestinationCount = 0`, so the final destination array consists solely of the fee leg, and `_fee = _totalOut`: the source is drained by the full amount and everything lands in the accumulator (tokenId 0). The caller's transaction succeeds but delivers nothing to any of their requested destinations. The entire withdrawal effectively becomes a burn/donation to the protocol.

Code Corrected
A `_nonZeroDestinationCount == 0` check has been added.

#011 `getPastVotes()` Accepts Future Timestamps

Informational	Version 1.1	Code Corrected
---------------	-------------	----------------

The function `VotingEscrow.getPastVotes()` does not reject `_timestamp` values in the future. For a future timestamp, it uses the latest (current) delegated voting power for the delegated portion, and for the self-held (non-delegated) portion it extrapolates the future non-permanent voting power forward along the decay curve.

Code Corrected
`getPastVotes()` has been updated to revert when the input `_timestamp` is larger than `block.timestamp`.

#012 `MAX_BASE_RATE_CHANGE_PIPS` Naming

Informational	Version 1.2	Code Corrected
---------------	-------------	----------------

In Minter, `MAX_BASE_RATE_CHANGE_PIPS` is a cap on `maxBaseRateChangePips` and so could be called `MAX_BASE_RATE_CHANGE_PIPS_CAP` or something that reflects its semantics.

Code Corrected
The suggestion was applied.

#013 Base Contract for Token and ReceiptToken

Informational	Version 1.2	Code Corrected
---------------	-------------	----------------

As it stands, a significant portion of `Token` and `ReceiptToken` contains the same logic. To reduce code duplication, a base contract could be implemented.

Code Corrected

A base contract definition was added.

#014 ChainStatus Default Value

Informational

Version 1.2

Code Corrected ✓

Since `Active` is the default value of `ChainStatus`, chains that have never been touched are active by default. This is not a problem in the current code because Voter always acts on chains that are both active and registered, but in general this is a fragile pattern.

Code Corrected

the default value of `ChainStatus` is now `ChainStatus.None`.

#015 Empty Rebalance Emits an Empty Event

Informational

Version 1.0

Acknowledged ⌵

`VotingEscrow.rebalanceUnderlying` does not require `_sources` or `_destinations` to be non-empty. When both arrays are empty, every loop is skipped, the conservation check passes trivially with `_totalIn` and `_totalOut` both zero, and the monotonic-unlock rule holds by construction. The forwarded call to `Voter.rebalanceChain0` returns immediately, as it explicitly tolerates an empty batch.

The call is therefore a no-op that changes no state and returns an empty `_mintedIds` array. It nevertheless emits `Rebalance([], [])`, with both event parameters empty, which carries no information for off-chain consumers.

Acknowledged

Client stated:

Intentional. The call is a harmless no-op; the cost of the extra input checks on every real call outweighs preventing an empty event that only the caller pays for.

#016 ERC721Enumerable Compatibility

Informational

Version 1.1

Code Corrected ✓

The sAERO implementation does not support the ERC-721 Enumerable extension at the moment. As you mentioned, this might be considered in the future. We want to highlight again that the current implementation of `totalSupply()` would be incompatible: it currently returns the total voting power, whereas the EIP expects `totalSupply()` to return the number of NFTs.

Code Corrected

The function has been updated to return the latest existing NFT ID, making it compliant with ERC721Enumerable. The bespoke functionality is now in `totalVotingPower()`.



#017 Gas Optimisations

Informational	Version 1.1	Code Corrected 
---------------	-------------	--

We make a non-exhaustive list of potential gas optimisations:

1. Redundant storage read in the binary-search loops of `BalanceLogicLibrary` : In both `getPastUserPointIndex` and `getPastGlobalPointIndex` , each iteration of the binary-search `while` loop reads the checkpoint timestamp from storage twice: once in the `userPoint.ts == _timestamp` comparison and again in the `userPoint.ts < _timestamp` comparison (and likewise `globalPoint.ts` in the global variant).
2. Storage read of a known-zero slot in `VotingEscrow.createStake()` : The function passes `_staked[_tokenId]` as the `_oldStaked` argument to `_depositFor` . At that point `_tokenId` is the freshly incremented `++tokenId` , so the `_staked[_tokenId]` slot has never been written and is necessarily zero-initialized. The value is therefore read from storage despite being known in advance.
3. In `VotingEscrow` , what purpose does the `DECIMALS` public constant (and its auto-generated `DECIMALS()` getter) serve? We could not find any use of it within the codebase. If possible, removing it could be considered a minor gas optimization.

Code Corrected

Items (1) and (2) were implemented. For (3), the unused DECIMALS constant was removed.

#018 Griefing rebalanceChain0 by Reshaping the Destination Stake

Informational	Version 1.2	Code Corrected 
---------------	-------------	--

`Voter._addChain0Contribution` reverts with `DstShapeStale` when the destination already holds chain0 allocation and the shape stored in the Voter no longer matches the live stake, that is `_dstHasVoted && tokenStates[_dst].lastStakeEnd != _dstStakeEnd` .

`VotingEscrow.increaseStakingPeriod` and `upgradeToPermanentStake` change the stake end without notifying the Voter, so `Voter.tokenStates[_dst].lastStakeEnd` goes stale. The holder of `_dst` can call either function to make any subsequent batch crediting that token revert. The entry point is `Voter.rebalanceChain0` , reached through `VotingEscrow.rebalanceUnderlying` , so the griefing blocks the `VoterPaymentsModule` deposit and withdrawal paths routing to that token until the holder votes again and re-syncs the stored shape.

Code Corrected

Every `VotingEscrow` path that reshapes a stake now synchronizes the Voter. `_depositFor` , which covers `increaseStakingPeriod` , along with `upgradeToPermanentStake` and `downgradeFromPermanentStake` , call `Voter.parkOnChain0` , and `withdraw` calls `Voter.clearToken` . `parkOnChain0` re-anchors a stale stored shape to the live one through `_reanchorToLiveShape` before adding the contribution, so `lastStakeEnd` can no longer drift out of sync and `_addChain0Contribution` no longer reverts for this reason.

#019 Incorrect Unit in `_creditDeallocation` Comment

Informational

Version 1.3

Code Corrected 

In `Voter`, `_credit` is describe as "Voting power actually credited to `CHAIN0`". It is not a voting power amount, but an AERO amount.

Code Corrected

The comment was rectified.

#020 Misleading Argument Name

Informational

Version 1.4

Open 

In `Migration`, the second argument to `_settleStake()` is called `_out`. This suggests that it is an amount in V3 currency since this is an output currency. However, at this point in the call path, the amount is still in V2 currency since the conversion has not been computed. In `depositVeNFT()` the variable is still called `_amount`.

The same applies to the second argument of `_settleLiquid()`.

The functionality is correct, this is purely a readability concern.

#021 Misleading Comment in `_allocationChainIds()`

Informational

Version 1.2

Code Corrected 

The `dev` comment before `Voter._allocationChainIds` states that the function `_applyChainAllocation` is the sole writer. This is not true since `burn`, `_removeChain0Contribution` and `_addChain0Contribution` also update the mapping.

Code Corrected

The comment was corrected in `Version 1.6`.

#022 Misleading Recipient in Rebalance Events

Informational

Version 1.0

Code Corrected 

The `recipient` field of `RebalanceDelta` is only acted upon when the leg's `tokenId` is the mint sentinel, where it becomes the owner of the freshly minted sAERO. On all other legs, meaning an add to an existing token, an accumulator leg, or any source leg, it is ignored by both `VotingEscrow` and `Voter.rebalanceChain0`.

The raw structs are however echoed into events by both contracts:

1. `VotingEscrow.rebalanceUnderlying` emits the unmodified calldata in `Rebalance` and forwards it to `Voter.rebalanceChain0`, which re-emits it in `Chain0Rebalanced`.

- 2. `VoterPaymentsModule.depositIntoNFT` echoes the caller-supplied sources in `DepositedIntoNFT` and passes `_recipient` through to the destination leg even when the destination is an existing token or the accumulator. `withdrawToNFT` copies `_destination.recipient` onto every trimmed destination leg and echoes the raw destinations in `WithdrawnToNFT`.

An event log can therefore show a non-zero `recipient` on a leg that credited an existing NFT owned by someone else entirely, which may mislead indexers and off-chain consumers.

A sanity check that non-mint legs carry no recipient, that is `tokenId != MINT_SENTINEL` implies `recipient == address(0)`, would keep inputs canonical and ensure the emitted events cannot carry misleading recipient data. The mint case needs no companion check, as a zero recipient on a mint leg already reverts inside the ERC-721 `_mint`.

Code Corrected

The struct `RebalanceDelta` has been split into `SourceDelta`, which no longer carries a `recipient` field, and `DestinationDelta`.

In `VotingEscrow`, `rebalanceUnderlying` now reverts with `NonMintRecipientNotAllowed` for any non-mint destination leg carrying a non-zero recipient.

In `VoterPaymentsModule`, `depositIntoNFT` overwrites the recipient with the zero address unless the destination is the mint sentinel, so the caller-supplied value is silently dropped rather than rejected. `withdrawToNFT` reverts with `IVotingEscrow.NonMintRecipientNotAllowed` when any caller-supplied destination violates the rule.

#023 OP Mainnet Naming

Informational	Version 1.4	Open
---------------	-------------	------

In `VelodormeMigration`, the network with chain ID 10 is referred to as "Optimism" in constants and comments. Since June 2023, this network is canonically called "OP Mainnet".

#024 Redundant Event Emissions

Informational	Version 1.1	Code Corrected
---------------	-------------	----------------

The following functions emit events unconditionally, even when the call does not change any state:

- 1. `VotingEscrow.burnFees()` : when `_amount == 0`, still emits `Supply` (re-emitting the unchanged value) and `Burn(0)`.
- 2. `VoterPaymentsModule.setFee()` : when re-set to the same rate, still emits `FeeSet`.
- 3. `VoterPaymentsModule.setRestricted()` : when set to the same flag value, still emits `RestrictedSet`.
- 4. `VotingEscrow.setArtProxy()` : when `_proxy` is already the same as `artProxy`.

Code Corrected

The code has been updated to prevent the redundant event emissions.

#025 Restricted Guard in removeFee

Informational

Version 1.0

Code Corrected 

`VoterPaymentsModule.removeFee` reverts with `RestrictedActive` when `restricted[_sig]` is set. The stated intent is to prevent the fee manager from locking a caller out of a restricted operation by deleting its registered entry, which is that caller's only access while the restriction is active.

The guard does not enforce this. `setRestricted` is gated by the same `FEE_MANAGER` role, so the fee manager can clear the flag, call `removeFee`, and set the flag again, all within a single transaction. The guard therefore only adds friction to a path that remains fully available.

Code Corrected

The `restricted[_sig]` check has been removed from `removeFee`. Client stated:

Agreed, we will fix it by dropping the guard. It is only friction, and removal from the mapping is a legitimate blacklist tool precisely when the function is restricted, so the revert contradicts the intended use. `removeFee` will succeed regardless of the restricted flag.

#026 Revoking the FED Freezes the Dynamic Rate Module

Informational

Version 1.2

Acknowledged 

`Minter.setDynamicRateModule` is restricted to `fed`. The governor can set `fed` to the zero address through `setFED`, which applies no zero check, after which no account can call `setDynamicRateModule` and `dynamicRateModule` retains its last value.

Revoking the FED therefore does not disable dynamic rates, it only removes the ability to change or clear the module. `emissionRate()` keeps consulting the previously set module for as long as a band is configured, and governance has no path to unset it.

Acknowledged

Velodrome stated that this behavior is intentional.

#027 Role Admins Can Be Revoked

Informational

Version 1.0

Code Corrected 

Some contracts in the codebase inherit from OpenZeppelin's `AccessControlEnumerable` and define roles that are their own admin.

For such a role, the only accounts allowed to grant it are its current holders. Neither `revokeRole()` nor `renounceRole()` is overridden, so the holders can remove themselves. Once the last holder is removed, the role can never be granted again.

This permanently strands the roles administered by it, as they can no longer be granted or revoked. The functions gated by the self-administering roles themselves also become unreachable.

Code Corrected

The abstract contract `GuardedAccessControlEnumerable` has been introduced and is inherited by the affected contracts. It overrides `_revokeRole()`, through which both `revokeRole()` and `renounceRole()` funnel, and reverts with `LastRoleHolder` when the removal would drop a self-administering role to zero holders. Roles that are administered by a different role remain fully revocable.

#028 Root and Leaf Chains Drift After Setting Them Both from Suspended to Active

Informational

Version 1.2

Acknowledged

When a chain is suspended and later reactivated, the root and leaf `Voter` contracts resume emissions asymmetrically: on reactivation the root immediately resumes accruing the chain's `ceiling` at its stored rate `R`, but the leaf's `chainRate` remains parked at `0` and is only restored when an inbound vote message next reaches the leaf via `applyChainState`. Crucially, reactivating the chain on root (`setChainStatus`) dispatches no cross-chain message, so nothing about the resume itself triggers this resync. It depends entirely on the next organic vote to that chain. If no such vote occurs, the gap is unbounded: the leaf indefinitely credits gauges nothing while the root keeps accruing redemption headroom at `R`, leaving the root's ceiling inflated relative to the emissions the leaf actually distributes.

Acknowledged

Client stated

This is documented design, not a bug: the NatSpec on `IVoter.setChainStatus` (updated in #505) sets the order, suspending the leaf first and resuming the root first so the drift window under-emits, with no funds at risk. The resync is not left to an organic vote, it is the binding last step of that checklist: a zero-delta `allocateChains` once the leaf is Active, which restores the scalar and whose nonce beats any straggler. It cannot happen inside `setChainStatus` because root and leaf resume on different chains, so a poke sent before the leaf is Active lands masked to zero. The only residue is governance skipping its own checklist, and the structural hardening is tracked separately.

#029 Unused Flash NFT Guard in balanceOfNFT

Informational

Version 1.4

Open

`VotingEscrow.balanceOfNFT` zeroes its return value for a `tokenId` that changed owner in the current block. The `_update` hook tags every real transfer, excluding mints and burns, in `ownershipChange`, and the getter returns zero while `ownershipChange[_tokenId] == block.number`.

No caller in scope reads `balanceOfNFT`. Voting weight and reward weight are derived from `staked()` snapshots and from `balanceOfNFTAt()`, neither of which applies the zero-out, so the guard protects no internal accounting.

Its only effect is on external integrators, for which the zero is misleading: `balanceOfNFTAt(_tokenId, block.timestamp)` returns the real voting power in the same transaction, so the two getters disagree for the duration of the transfer block.



#030 Zero Amounts in rebalanceDelta

Informational	Version 1.1	Acknowledged 
---------------	-------------	--

`VoterPaymentsModule.depositIntoNFT` does not reject source entries with `amount = 0`. Such a leg passes through `VotingEscrow.rebalanceUnderlying` and `_applySource` without effect: the stake balance, `permanentStakeBalance`, and the scheduled slope changes are all rewritten with their exact prior values, so no value-level state changes and no view function returns a different result.

The call is however not a complete no-op at the storage level. Each zero-amount leg still runs the full checkpoint machinery, performing redundant SLOADs/SSTOREs of unchanged values, and appends a new user point for the source token that is identical to the previous one except for its timestamp (along with a `MetadataUpdate` event despite no metadata change).

The impact is limited to redundant gas consumption paid by the caller and gradual growth of the per-token point history with duplicate entries.

A similar laxity exists in `withdrawToNFT` on the destination side: individual destination entries with `amount = 0` are accepted, as only the destination total is required to be nonzero. These legs are dropped by the proportional-trim filter before reaching the escrow, so they cause no checkpoint writes. The effect is limited to wasted gas and the zero entries being echoed in the `WithdrawnToNFT` event.

Acknowledged

Client stated:

Intentional: the gas is paid by the caller who supplied the useless legs, the duplicate checkpoint entries are mathematically inert, and rejecting them adds checks to every real call. Not worth hardening.

8 Open Questions

Here, we list open questions that came up during the assessment and that we would like to clarify to ensure that no important information is missing.

OQ1 Recipient Can Be the Migration Contract Itself

Open Question	Version 1.4
---------------	-------------

In the base migration contract, the caller of `depositVeNFT` can set `_recipient` to any non-zero address. The zero address is the only value rejected.

In particular, `_recipient` can be set to the address of `VelodromeMigrationEntrypoint`, which is also the address of `VelodromeMigration`, as `handle` requires the cross-chain sender to equal the entrypoint's own address. In this case the v3 TOKEN is locked on `VelodromeMigrationEntrypoint` if the position was liquid. For illiquid positions the v3 sTOKEN is locked on the contract instead.

Also, by design of `AerodromeMigration.onERC721Received`, which accepts any `V3_ESCROW` transfer the contract itself operates, sTOKEN can similarly be sent to `AerodromeMigration`.

Since the design of this function suggests this is a deliberate choice, we wanted to ask if this was indeed intentional.

OQ2 Root Domain in VelodromeMigration

Open Question	Version 1.4
---------------	-------------

The root domain in `VelodromeMigration` is always supposed to be Base. `ROOT_DOMAIN` is nevertheless a constructor argument, validated only as non-zero and different from the mailbox's local domain, and every deployment is expected to pass the same value.

Did you consider setting `ROOT_DOMAIN` to the constant `8453`?



9 Limitations and use of report

Security assessments cannot uncover all existing vulnerabilities; even an assessment in which no vulnerabilities are found is not a guarantee of a secure system. However, code assessments enable the discovery of vulnerabilities that were overlooked during development and areas where additional security measures are necessary. In most cases, applications are either fully protected against a certain type of attack, or they are completely unprotected against it. Some of the issues may affect the entire application, while some lack protection only in certain areas. This is why we carry out a source code assessment aimed at determining all locations that need to be fixed. Within the customer-determined time frame, ChainSecurity has performed an assessment in order to discover as many vulnerabilities as possible.

The focus of our assessment was limited to the code parts defined in the engagement letter. We assessed whether the project follows the provided specifications. These assessments are based on the provided threat model and trust assumptions. We draw attention to the fact that due to inherent limitations in any software development process and software product, an inherent risk exists that even major failures or malfunctions can remain undetected. Further uncertainties exist in any software product or application used during the development, which itself cannot be free from any error or failures. These preconditions can have an impact on the system's code and/or functions and/or operation. We did not assess the underlying third-party infrastructure which adds further inherent risks as we rely on the correct execution of the included third-party technology stack itself. Report readers should also take into account that over the life cycle of any software, changes to the product itself or to the environment in which it is operated can have an impact leading to operational behaviors other than those initially determined in the business specification.